GitHub Issues
A working guide

How the A2B project tracks what is being built, what is broken, and what was decided - and how to read it without being a developer.

THE LABELS ON THIS BOARD

p0-blocker p1 p2 epic bug area:runtime area:infra area:console area:publish area:ingest area:content area:tenancy area:compiler

ALL-Dunkin / A2B-system

15 open issues · 11 closed · 11 August 2026

SECTION ONE

What an issue actually is

One idea, and the rest of this guide is mechanics.

THE POINT

An issue is one piece of work with a permanent address and its own conversation attached. That is all. Its value is not that it is a to-do item - it is that six months from now you can find out **why** something was done, and by whom, without asking anyone.

Every team already tracks work somewhere: a spreadsheet, a chat thread, somebody's memory. Those all work until the moment you need the reasoning behind a decision, at which point the spreadsheet has a row that says "done" and the chat thread has scrolled away.

Issues fix that by putting the discussion in the same place as the task, and never deleting either.

What you get that a spreadsheet cannot give you

	Spreadsheet or chat	GitHub Issues
Finding it later	Search your memory for keywords	Permanent number: #24, forever
Why was this done?	Ask whoever remembers	Read the comments on the issue
Is it finished?	Someone updates a cell, or forgets	Closes automatically when the code ships
What changed the code?	No connection at all	The commit is linked to the issue
Status meeting	Ask each person in turn	One saved search, read in 30 seconds

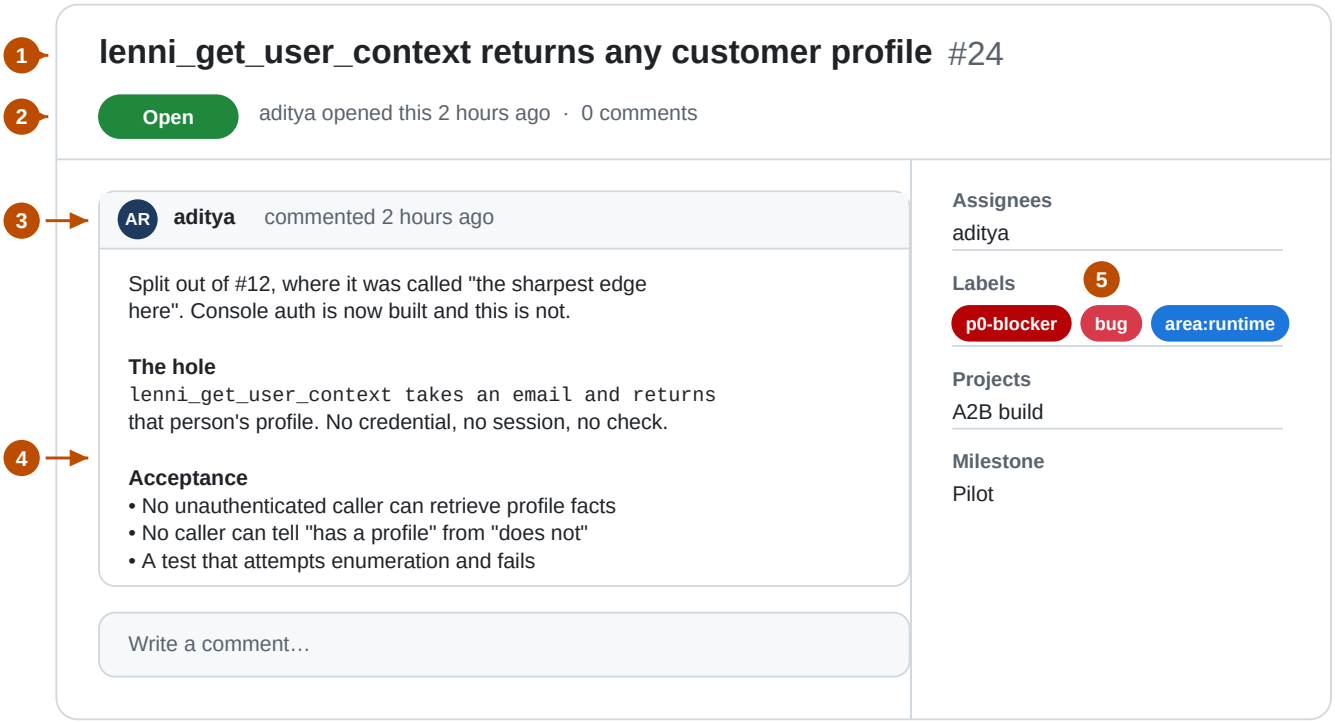
The habit that matters most

If a decision gets made about a piece of work - in a meeting, in a hallway, on a call - it goes in a comment on the issue. Not because of process, but because the person who needs it six months from now will not have been in that meeting, and will not know to ask.

SECTION TWO

Anatomy of an issue

This is issue #24 on your board, drawn to scale. Five things to know.



A real issue from this project. The numbered markers are explained below and are not part of the interface.

1. **Title and number.** The number (**#24**) is permanent and is how everyone refers to this piece of work in conversation, in commit messages and in other issues. It is never reused, even after the issue is closed.
2. **State. Open** or **Closed**, and nothing in between. If you need more states than that - in review, blocked, in progress - that is what Projects are for, in section eleven.
3. **Who and when.** Every action is attributed and timestamped, automatically. Nobody has to remember to write a status update for this to be true.
4. **The body.** Written by whoever opened it, and editable later. Aim for: what is wrong, why it matters, and how we will know it is fixed. The **Acceptance** section at the bottom is that last part.
5. **Labels, assignee, milestone.** The filing system, down the right-hand side. Labels are the important one and get a section of their own. An issue with no labels is invisible to every useful search.

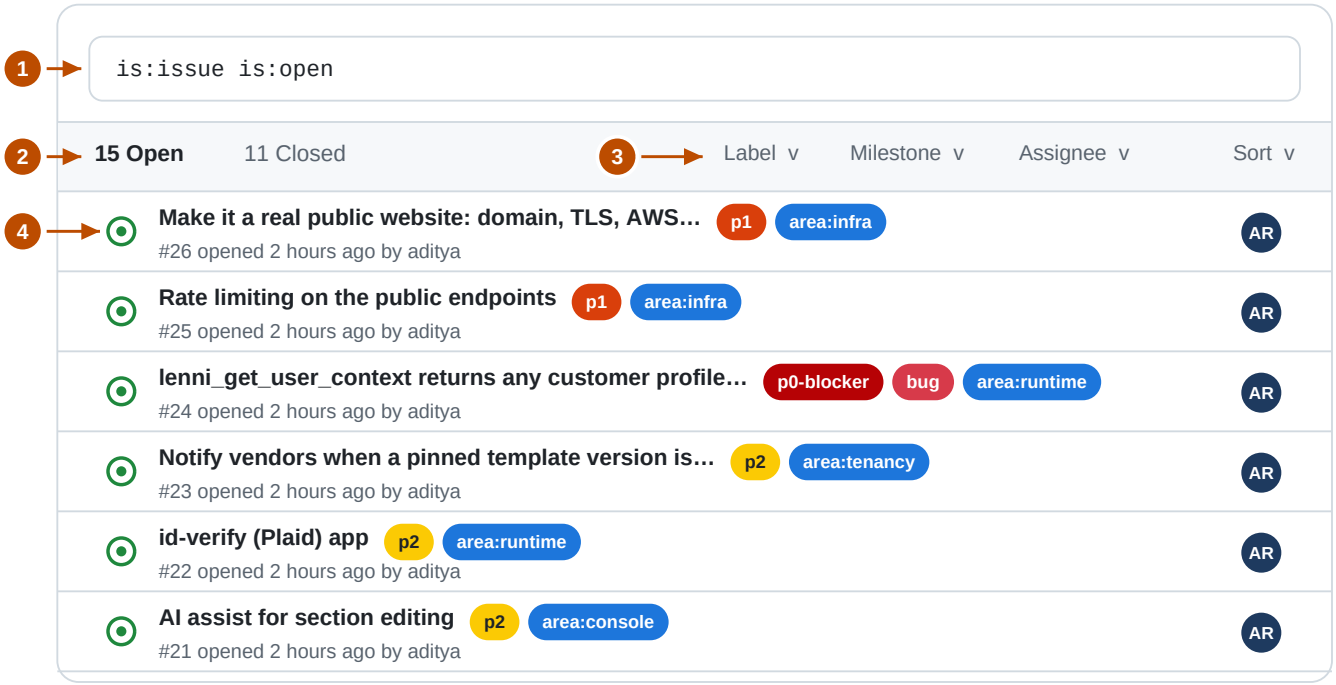
What you can safely ignore

An issue page has more controls than anyone uses. Reactions, subscribing, pinning, transferring, locking - none of them matter for reading the board. Four things carry all the meaning: the **title**, the **labels**, whether it is **open**, and the **comments**. If you read only those you will not miss anything important.

SECTION THREE

Anatomy of the list

The screen you will spend the most time on. It is a search engine, not a folder.



The Issues tab, showing the top of the real board.

1. **The search box is the whole interface.** It is not a text search, it is a query language. `is:issue is:open` is the default and gives the list shown here; adding `label:p0-blocker` narrows it to "open issues that block deployment". Section seven has the five queries worth bookmarking.
2. **Open and Closed are two separate lists.** Closed issues are not deleted and not hidden - they are the record of what was done and why. Click **Closed** to read the project's history.
3. **Filter menus write the query for you.** If you would rather not learn the query syntax, these dropdowns build it. The box updates as you click, which is a decent way to learn it.
4. **One row per issue.** The coloured dot means open. The labels sit next to the title so priority is visible without opening anything - which is the entire point of labelling consistently.

THE POINT

If you learn one thing from this guide, learn that the box at the top is a query. Typing `is:open label:p0-blocker` answers "what is stopping us shipping?" in one line, and you can bookmark the result.

SECTION FOUR

Your board, right now

All 15 open issues, most urgent first. This is the table the rest of the guide teaches you to read.

#	Priority	Area	What it is
24	p0-blocker	area:runtime	lenni_get_user_context returns any customer profile given its email
26	p1	area:infra	Make it a real public website: domain, TLS, AWS networking
25	p1	area:infra	Rate limiting on the public endpoints
18	p1	area:publish	Establish which terminal deep links actually route
17	p1	area:ingest	Multi-page ingest: crawl beyond a single URL
16	p1	area:content	Expand the product registry to full breadth
15	p1	area:infra	Outbound email: team invites and the email destination
14	p1	area:runtime	Convey delivery integration
13	p1	area:runtime	Port FFIEC bank matching as the fdic-bank-data app
23	p2	area:tenancy	Notify vendors when a pinned template version is superseded
22	p2	area:runtime	id-verify (Plaid) app
21	p2	area:console	AI assist for section editing
20	p2	area:console	Funnel analytics: aggregate the events into something readable
19	p2	area:infra	Migrate from SQLite to Postgres
1	epic		Epic: A2B system - build status and roadmap

Read it top to bottom and the state of the project is visible: one thing blocks deployment (#24), a cluster of work is needed before a real pilot, and the rest is wanted but not urgent. The eleven closed issues are the history and are worth reading once.

SECTION FIVE

Labels: the filing system

Two axes, deliberately. Priority answers "in what order", area answers "which part of the system".

Priority - how urgent

Label	Means	In practice
p0-blocker	Blocks deployment	Drop other things. There is one right now: #24.
p1	Needed for a real pilot	Must be done before a customer sees this.
p2	Wanted, not urgent	Real work, no date. Safe to leave alone.
epic	Tracking issue	Not work itself - a summary of other issues.

Area - which part of the system

All eight are the same blue on purpose: they are categories, not priorities. Colour is reserved for urgency, so a glance at the board reads as urgency rather than as decoration.

area:runtime	Sessions, chaining, MCP
area:infra	Deployment, storage, auth
area:console	Vendor console
area:publish	Links and the agent manifest
area:ingest	Website scanning and matching
area:content	Registry: products, templates, library
area:tenancy	Vendors, skills, version pinning
area:compiler	Prompt compilation

Why this is worth being strict about

An issue with no labels does not show up in any of the searches in section seven. It is not lost exactly, but nobody will see it again unless they scroll past it. Two labels - one priority, one area - takes three seconds and is the difference between a board and a pile.

THE POINT

Every issue gets exactly one priority label and one area label. Extras like `bug` are fine on top; those two are the ones that make it findable.

SECTION SIX

Writing one worth reading

The same problem, filed two ways. The difference is about ninety seconds of writing.

Hard to act on

Login is broken

no labels · no assignee

it doesnt work when i tried it
can someone look

Why this costs a day:
- Which login? Console or customer?
- Broken how - an error, a blank page, or a wrong password accepted?
- No labels, so it appears in no search
- Nobody can tell when it is fixed

Someone has to ask three questions before work can start, and the answers land in chat instead of here.

Ready to pick up

Login accepts an expired session cookie

p0-blocker bug area:infra · assigned

Steps to reproduce
1. Log in, then wait past expiry
2. Reload /console/

Expected: sent back to the login page.
Actual: still signed in.

Where I think it is
`auth.user_for_token compares expires_at as a string.`

Acceptance
- An expired cookie is rejected
- A test that fails before the fix

Both describe a real bug. Only one can be picked up without a conversation first.

The three questions to answer

Question	Why it matters
What is wrong?	Specifically, and in the title. "Login is broken" could be six different bugs; the title should say which.
Why does it matter?	Whoever prioritises this is not necessarily the person who found it. Say what breaks and for whom.
How will we know it is fixed?	This is the one people skip, and the one that prevents an issue sitting open for a month because nobody agreed what done meant.

You are not expected to know the technical cause. "Clicking the Claude button does nothing" is an excellent issue - it says what was clicked and what failed to happen, which is everything needed to start.

SECTION SEVEN

Five searches worth bookmarking

Paste into the search box on the Issues tab, then bookmark the resulting page.

Question	Query
What is blocking us right now?	`is:open label:p0-blocker`
What must happen before a pilot?	`is:open label:p1`
What shipped in the last week?	`is:closed closed:>2026-08-04`
What is nobody working on?	`is:open no:assignee`
What has gone quiet?	`is:open updated:<2026-07-14`

How to run a status check without interrupting anyone

Open the first two bookmarks. The first tells you whether anything is on fire; the second tells you how far the pilot is. If an issue looks stalled, read its comments before asking - the answer is usually already there, which is the whole reason for writing them.

A few more pieces of syntax

`is:open`	Not yet finished
`is:closed`	Finished, kept as record
`label:p1`	Has that label
`-label:p2`	Does **not** have it - note the minus
`assignee:aditya`	Assigned to a person
`no:label`	Never filed properly - worth checking
`sort:updated-desc`	Most recently touched first

Terms combine with spaces, and spaces mean "and". So `is:open label:p1 no:assignee` is "needed for the pilot and nobody has picked it up" - which is usually the most useful question a manager can ask.

SECTION EIGHT

Comments are the decision log

The part that pays for itself, and the part teams skip.

When something is learned about a piece of work, it goes in a comment rather than replacing the original text. The original stays wrong on purpose - the sequence of what was believed, then discovered, then decided is the useful part.

A real example from this board

Issue #12 asked for authentication. It was written when nothing was deployed and it argued that shipping half-built authentication would be worse than shipping none. Months later the work was done - but not all of it. Rather than quietly closing it, a comment recorded what was built, which two decisions were made and why, and which two pieces of the original scope were **not** done. Those two became #24 and #25.

> THE POINT
>
> Because that comment exists, nobody has to wonder whether the missing pieces were forgotten or deferred on purpose. That distinction is invisible in a spreadsheet and it is the most expensive thing to get wrong.

What is worth a comment

- A decision, and the reason for it - especially rejecting the obvious approach, because the next person will suggest it again.
- Something that turned out to be wrong. #18 was written believing Claude could not accept a prefilled prompt; that had changed. The comment says so, so the belief does not outlive the fact.
- Scope that will not be done, and where it went instead.
- What was verified, and what was not. If you tested it in one browser and not another, say which.

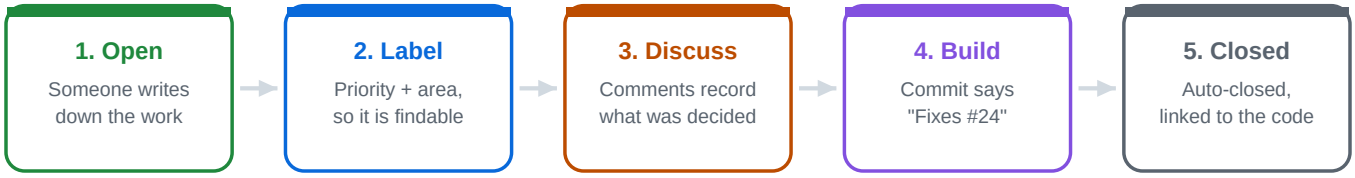
What is not

"Any update?" - the timeline already shows when it was last touched. If it has gone quiet, ask the person directly and put the answer in a comment.

SECTION NINE

From opened to closed

What happens to an issue, and the one bit of magic worth knowing.



The whole point of step 4:
writing "Fixes #24" in a commit or pull request closes the issue and links the code that fixed it, permanently.

Closing is not deleting

A closed issue stays readable forever, keeps its number, and keeps its comments. Closing means "no longer needs attention", not "gone". The eleven closed issues on this board are the project's history, and reading them is the fastest way for someone new to understand how it got here.

Linking code to the reason for it

When the work is done, whoever did it writes `Fixes #24` in the commit or pull request. GitHub then closes the issue and links the two permanently. A year later, anyone looking at that code can get from a confusing line to the discussion that produced it in two clicks.

This is the single highest-value habit in this guide and it costs eight characters.

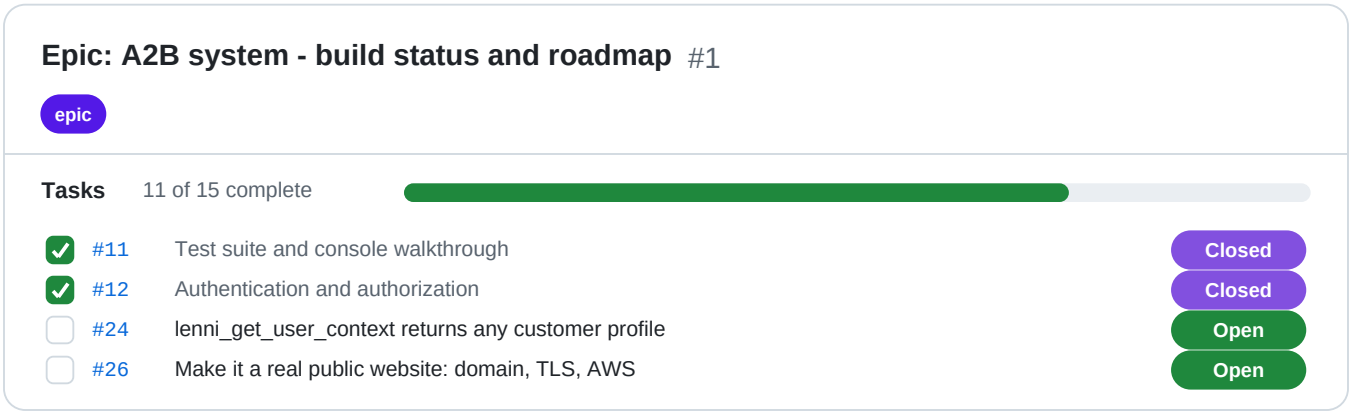
If it turns out not to be a real problem

Close it with a comment saying why, or label it `wontfix`. Both are better than leaving it open, because an open issue nobody intends to do makes every count on the board a lie.

SECTION TEN

Epics: one issue that watches the others

When a project has more issues than fit in your head.



Issue #1 on this board. Writing #24 inside a checklist turns it into a live row: the title, the state and the progress bar all update themselves.

An epic is a normal issue that holds a checklist of other issues. Its job is to be the one link you send someone who asks "how is the project going?". It does not track work of its own.

Why it is worth the effort

- The progress bar is computed, not maintained. Nobody updates it and so nobody can forget to.
- It gives newcomers a front door. #1 on this board opens with what A2B is, then what is built, then what is not.
- It survives people leaving. The reading order lives in the issue rather than in whoever has been here longest.

THE POINT

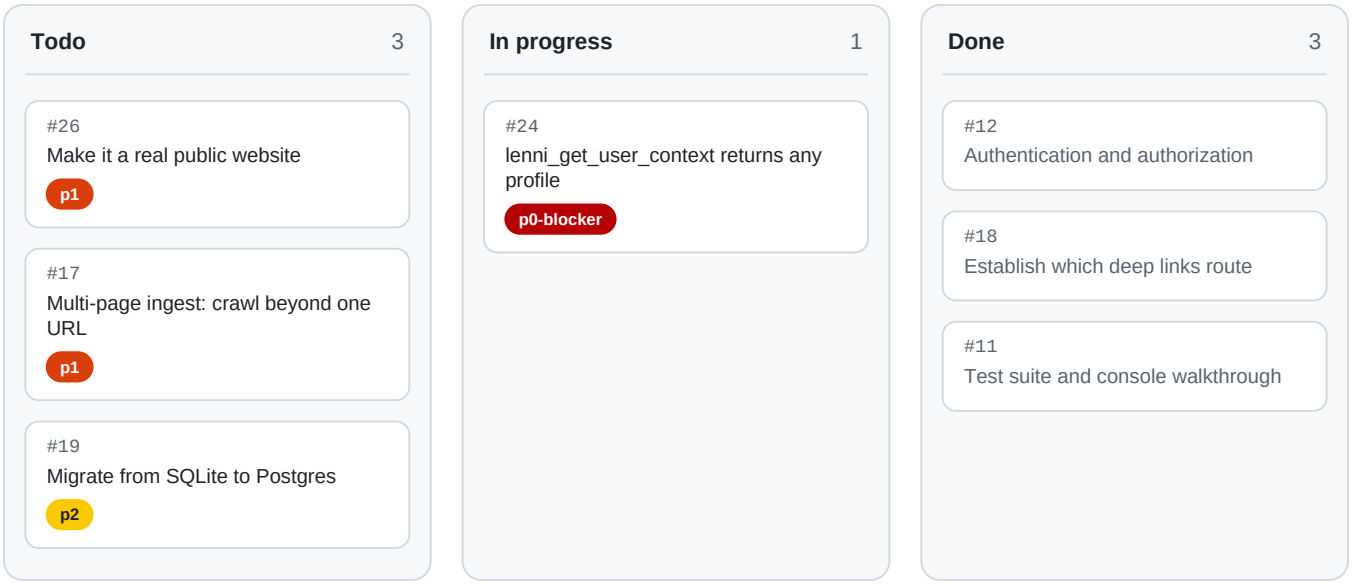
If you read exactly one thing on this repository, make it issue #1. It is written to be that thing.

SECTION ELEVEN

Projects, and when you need one

Issues have two states. Real work has more. That gap is what Projects fill.

A Project is a board or table view built on top of the same issues - the classic columns of Todo, In progress, Done. Nothing is duplicated: an issue on a project board is the same issue, and moving the card does not change the issue.



Issues from this project, arranged as a board. Compare it with section three: identical issues, different question answered.

Use	When
The Issues list	You want to know what exists and how urgent it is. This is enough for most of what this project needs today.
A Project board	You care about flow - what is in progress right now, what is waiting on review, what is stuck. Useful with several people working at once.

The honest recommendation for this project: labels and saved searches are doing the job. Add a Project when more than one person is working at a time and "who is on what this week" stops being obvious - not before, because a board nobody drags cards on is worse than no board, since it looks authoritative while being stale.

Milestones

A milestone is a named bucket with an optional date - "Pilot", "v1". It gives you a percentage-complete for a release. Worth adding when there is a date to hit; noise before that.

SECTION TWELVE

One-page cheat sheet

The whole guide, for pinning up.

Opening an issue

- Title says what is wrong, specifically.
- Body: what is wrong, why it matters, how we will know it is fixed.
- One priority label and one area label. Always.
- Assign it if you know who; leave it unassigned if you do not - `no:assignee` is a useful search.

Working on one

- Decisions go in comments, not in edits to the original text.
- Write `Fixes #24` in the commit or pull request.
- If scope drops out, open a new issue and say so in a comment.

Reading the board

- `is:open label:p0-blocker` - is anything on fire?
- `is:open label:p1` - how far is the pilot?
- `is:open no:assignee` - what has nobody picked up?
- Read the comments before asking for an update.

Keyboard shortcuts on github.com

`g i`	Go to Issues
`c`	Create a new issue
`/`	Jump to the search box
`l`	Edit labels on an open issue
`?`	Show every shortcut

Vocabulary

Issue	One piece of work, with a number and a conversation.
Label	A coloured tag. Here: one priority plus one area.
Epic	An issue holding a checklist of other issues.
Milestone	A named bucket of issues, usually with a date.
Pull request	Proposed code changes, reviewed before merging. Often says *Fixes #24*.
Commit	One saved change to the code, with a message.
Repository	The project: its code, its issues, its history.

Generated from the live board on 11 August 2026. Rebuild with `python scripts/build_issues_guide.py`.